

Практическое занятие №6: Полный цикл разведочного анализа данных (EDA) на реальном датасете

Цель занятия:

Применить на практике полный цикл разведочного анализа данных (EDA) к реальному набору данных. Научиться формулировать гипотезы, проверять их с помощью визуализации и статистических методов, а также представлять результаты анализа в структурированном виде.

Задачи:

1. Провести первичный анализ структуры и качества данных.
2. Выполнить очистку данных (обработка пропусков, дубликатов, выбросов).
3. Создать новые признаки на основе существующих.
4. Провести исследовательский анализ с помощью визуализации.
5. Сформулировать выводы и гипотезы на основе проведенного анализа.

Теоретические основы

Что такое разведочный анализ данных (EDA)?

EDA (Exploratory Data Analysis) - это первичный этап работы с данными, целью которого является изучение их структуры, выявление закономерностей, аномалий и формулирование гипотез. Это процесс «знакомства» с данными перед построением более сложных моделей.

Основные этапы EDA:

1. **Сбор и первичный осмотр (Data Collection & Initial Inspection):**
 - Загрузка данных из различных источников (CSV, Excel, базы данных).
 - Просмотр первых и последних строк (`head()`, `tail()`).
 - Получение общей информации о датасете (`info()`, `shape`, `columns`).
2. **Очистка данных (Data Cleaning):**
 - **Обработка пропусков:** выявление и заполнение/удаление пропущенных значений (`isnull()`, `fillna()`, `dropna()`).
 - **Удаление дубликатов:** поиск и удаление повторяющихся строк (`duplicated()`, `drop_duplicates()`).
 - **Работа с выбросами:** выявление аномальных значений с помощью визуализации (`boxplot`) и статистических методов (IQR), принятие решения об их обработке.

3. Преобразование и создание признаков (Feature Engineering):

- Приведение типов данных к корректным (`astype()`).
- Создание новых признаков на основе существующих (например, возраст из даты рождения).

4. Исследовательский анализ и визуализация (Exploration & Visualization):

- **Анализ распределений:** гистограммы, ящики с усами (`histplot`, `boxplot`).
- **Анализ взаимосвязей:** точечные диаграммы, тепловые карты корреляций (`scatterplot`, `heatmap`).
- **Сравнение групп:** столбчатые диаграммы, скрипичные графики (`barplot`, `violinplot`).

5. Формулирование выводов и гипотез (Conclusion & Hypothesis):

- Обобщение ключевых инсайтов.
- Постановка гипотез для дальнейшего анализа или моделирования.

Кроме того, в соответствии с современным подходом к обработке данных, полный цикл включает пять этапов:

Этап	Название	Что делаем	Ключевые методы pandas
1	Discover (Обнаружение)	Загружаем данные, изучаем структуру, находим проблемы	<code>.info()</code> , <code>.head()</code> , <code>.describe()</code> , <code>.isnull().sum()</code>
2	Structure (Структурирование)	Переименовываем столбцы, меняем типы, преобразуем форматы	<code>.rename()</code> , <code>astype()</code> , <code>pd.to_datetime()</code>
3	Clean (Очистка)	Удаляем дубликаты, заполняем пропуски, обрабатываем выбросы	<code>.drop_duplicates()</code> , <code>.fillna()</code> , <code>.dropna()</code>
4	Enrich (Обогащение)	Создаём новые признаки, объединяем данные	<code>.apply()</code> , <code>lambda</code> , <code>groupby()</code>
5	Validate (Валидация)	Проверяем результат: нет ли пропусков, корректны ли диапазоны	<code>.isnull().sum()</code> , проверка границ

Ключевые методы pandas для каждого этапа

1. Обнаружение проблем

- `df.info()` — выводит тип данных и количество непустых значений в каждом столбце.
- `df.describe(include='all')` — статистика для числовых и категориальных столбцов.
- `df.isnull().sum()` — количество пропусков в каждом столбце.
- `df.duplicated().sum()` — общее количество дублированных строк.

2. Работа с пропусками (метод `fillna`)

Метод `fillna()` позволяет заменить пропуски:

- **Константой:** `df.fillna(0)` — заменяет все пропуски нулями.
- **Статистикой по столбцу:**
`df['age'].fillna(df['age'].median())` — медиана лучше среднего, если есть выбросы.
- **Прямой/обратной заливкой (`ffill/bfill`):**
`df.fillna(method='ffill')` — для временных рядов, когда следующее значение похоже на предыдущее.
- **Словарём для разных столбцов:** `df.fillna({'price': 0, 'category': 'Unknown'})` — максимальный контроль.
- **Удалением строк с пропусками:** `df.dropna()` — если пропусков мало и их удаление не исказит данные.
- **Удалением столбца с большим количеством пропусков:**
`df.drop(columns=['column_name'])` — если пропущено >70% данных.

3. Работа с дубликатами

Метод `drop_duplicates()` позволяет гибко управлять дубликатами:

- `df.drop_duplicates()` — удаляет дубликаты, оставляя первое вхождение.
- `df.drop_duplicates(keep='last')` — оставляет последнее вхождение.
- `df.drop_duplicates(keep=False)` — удаляет все дубликаты.
- `df.drop_duplicates(subset=['column1', 'column2'])` — ищет дубликаты только по указанным столбцам.

4. Обработка выбросов методом межквартильного размаха (IQR)

Межквартильный размах (IQR) — надёжный статистический метод обнаружения выбросов:

$Q1$ = 25-й перцентиль (первый квартиль)

$Q3$ = 75-й перцентиль (третий квартиль)

$IQR = Q3 - Q1$

Нижняя граница = $Q1 - 1.5 \times IQR$

Верхняя граница = $Q3 + 1.5 \times IQR$

Выбросы — точки ниже нижней или выше верхней границы

Практическая реализация:

```
Q1 = df['column'].quantile(0.25)
```

```
Q3 = df['column'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower = Q1 - 1.5 * IQR
```

```
upper = Q3 + 1.5 * IQR
```

```
outliers = df[(df['column'] < lower) | (df['column'] > upper)]
```

```
df_clean = df[(df['column'] >= lower) & (df['column'] <= upper)]
```

5. Создание новых признаков

- **С помощью apply и lambda:** `df['new_col'] = df['old_col'].apply(lambda x: x * 2).`
- **С помощью groupby и transform:** `df['group_mean'] = df.groupby('category')['value'].transform('mean').`
- **Условное создание:** `df['age_group'] = df['age'].apply(lambda x: 'young' if x < 30 else 'old').`

Принципы безопасной работы с данными на общих компьютерах

- Всегда сохраняйте исходные данные отдельно от обработанных.
- Используйте `inplace=False` (по умолчанию) при экспериментировании, чтобы не испортить исходный DataFrame.
- Проверяйте результат после каждого шага очистки.
- После завершения работы очищайте историю команд в терминале и удаляйте кэш учётных данных, если использовали Git (см. предыдущие рекомендации по Git).

Подробный пример: полный цикл обработки датасета «Титаник»

Для демонстрации всех этапов используем классический датасет о пассажирах «Титаника».

Шаг 1. Загрузка и первичный анализ (Discover)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Загружаем данные
df = sns.load_dataset('titanic')
print("Размер исходных данных:", df.shape)
print("\nПервые 5 строк:")
df.head()
```

Результат:

- `df.info()` показывает, что в столбце `age` 177 пропусков, в `deck` — 688 пропусков, в `embark_town` — 2 пропуска.
- `df.describe()` показывает, что средний возраст ~30 лет, а максимальная стоимость билета >500.

Шаг 2. Структурирование данных (Structure)

Приводим типы данных к нужному формату:

```
# Преобразуем столбец survived в категориальный
df['survived'] = df['survived'].astype('category')

# Проверяем, есть ли столбцы с датами (в Титанике нет, но для примера)
# df['date'] = pd.to_datetime(df['date_column'],
# errors='coerce')

# Переименовываем столбцы для удобства
df.rename(columns={'sibsp': 'siblings_spouses', 'parch':
'parents_children'}, inplace=True)
```

Шаг 3. Очистка данных (Clean)

3.1. Удаление столбца с большим количеством пропусков

```
# deck пропущен у 77% пассажиров — удаляем
df.drop(columns=['deck'], inplace=True)
print("Столбец deck удалён. Новый размер:", df.shape)
```

3.2. Обработка пропусков в столбце `embark_town`

```
# Всего 2 пропусков — удаляем строки
df.dropna(subset=['embark_town'], inplace=True)
print(f"После удаления строк с пропущенным портом: {df.shape}")
```

3.3. Заполнение пропусков в возрасте

```
# Медианный возраст по группам (класс + пол)
df['age'] = df.groupby(['pclass', 'sex'])['age'].transform(
    lambda x: x.fillna(x.median())
)
print(f"Пропусков в возрасте осталось:
{df['age'].isnull().sum()}")
```

3.4. Удаление дубликатов

```
duplicates_before = df.duplicated().sum()
print(f"Дубликатов до удаления: {duplicates_before}")

df.drop_duplicates(inplace=True)

duplicates_after = df.duplicated().sum()
print(f"Дубликатов после удаления: {duplicates_after}")
```

3.5. Обработка выбросов (на примере стоимости билета `fare`)

```
# Визуализация для обнаружения выбросов
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
df['fare'].plot(kind='box', title='Boxplot стоимости билета')
plt.subplot(1, 2, 2)
df['fare'].hist(bins=50, title='Гистограмма стоимости билета')
plt.tight_layout()
plt.show()

# Обнаружение выбросов методом IQR
Q1 = df['fare'].quantile(0.25)
Q3 = df['fare'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers_fare = df[(df['fare'] < lower) | (df['fare'] > upper)]
print(f"Количество выбросов в fare: {len(outliers_fare)}")
print(f"Границы: нижняя = {lower:.2f}, верхняя = {upper:.2f}")

# В учебных целях оставляем выбросы, но фиксируем факт
# В реальном проекте можно удалить или ограничить
# df = df[(df['fare'] >= lower) & (df['fare'] <= upper)]
```

Шаг 4. Обогащение данных (Enrich)

Создаём новые признаки для анализа:

```
# 1. Размер семьи
df['family_size'] = df['siblings_spouses'] +
df['parents_children'] + 1

# 2. Путешествовал ли один
df['is_alone'] = (df['family_size'] == 1).astype(int)

# 3. Возрастная группа
def age_group(age):
    if age < 18:
        return 'child'
    elif age < 60:
        return 'adult'
    else:
        return 'elderly'

df['age_group'] = df['age'].apply(age_group)

# 4. Категория стоимости билета
df['fare_category'] = pd.cut(df['fare'],
                             bins=[0, 10, 30, 100, 600],
                             labels=['economy', 'standard',
                                     'premium', 'luxury'])
```

Шаг 5. Валидация (Validate)

Проверяем, что все операции выполнены корректно:

```
print("=== Финальная проверка ===")
print(f"Размер итогового датасета: {df.shape}")
print(f"Пропуски в каждом столбце:\n{df.isnull().sum()}")
print(f"Уникальные значения в age_group:
{df['age_group'].unique()}")
print(f"Уникальные значения в fare_category:
{df['fare_category'].cat.categories}")

# Сохраняем результат
df.to_csv('titanic_cleaned.csv', index=False)
print("Датасет сохранён в файл titanic_cleaned.csv")
```

Итоговый пример полного пайплайна

```
def full_preprocessing_pipeline(df):
    """
    Полный пайплайн предобработки данных
    """
    # 1. Structure: переименование и типы
    df = df.copy()
```

```

df.rename(columns={'sibsp': 'siblings_spouses', 'parch':
'parents_children'}, inplace=True)

# 2. Clean: удаление столбца с пропусками
if 'deck' in df.columns:
    df.drop(columns=['deck'], inplace=True)

# 3. Clean: удаление строк с пропущенным портом
df.dropna(subset=['embark_town'], inplace=True)

# 4. Clean: заполнение возраста
df['age'] = df.groupby(['pclass', 'sex'])['age'].transform(
    lambda x: x.fillna(x.median())
)

# 5. Clean: удаление дубликатов
df.drop_duplicates(inplace=True)

# 6. Enrich: создание новых признаков
df['family_size'] = df['siblings_spouses'] +
df['parents_children'] + 1
df['is_alone'] = (df['family_size'] == 1).astype(int)
df['age_group'] = df['age'].apply(
    lambda x: 'child' if x < 18 else ('adult' if x < 60 else
'elderly')
)

return df

# Применение пайплайна
df_original = sns.load_dataset('titanic')
df_cleaned = full_preprocessing_pipeline(df_original)
print(f"Исходный размер: {df_original.shape}")
print(f"Размер после очистки: {df_cleaned.shape}")

```


Варианты для самостоятельного выполнения (10 вариантов)

Для каждого варианта студент должен выполнить полный цикл предобработки данных в соответствии с описанными этапами. Варианты различаются датасетами и специфическими проблемами данных, что требует разных подходов к очистке.

Общие требования для всех вариантов:

1. Загрузить указанный датасет.
2. Выполнить все 5 этапов: Discover → Structure → Clean → Enrich → Validate.
3. Создать минимум 2 новых признака.
4. Сохранить очищенный датасет в файл `cleaned_название_датасета.csv`.
5. Подготовить краткий отчёт (в Jupyter Notebook с Markdown-ячейками) с пояснением каждого шага.

Вариант 1: California Housing

- **Источник:** `from sklearn.datasets import fetch_california_housing` (или `sns.load_dataset`)
- **Особенности данных:** Цены на жильё в Калифорнии, есть выбросы по стоимости и количеству комнат.
- **Задачи:**
 1. Обработать пропуски (если есть).
 2. Обнаружить и обработать выбросы в столбце `MedHouseVal` методом IQR.
 3. Создать новый признак `rooms_per_household = AveRooms / AveOccup`.
 4. Нормализовать столбец `MedInc` (масштабирование от 0 до 1).
 5. Удалить дубликаты.

Вариант 2: Diabetes (диабет)

- **Источник:** `from sklearn.datasets import load_diabetes`
- **Особенности данных:** Медицинские показатели пациентов, все признаки уже нормализованы.
- **Задачи:**
 1. Преобразовать целевую переменную в категориальную (разбить на 3 группы: низкий, средний, высокий уровень прогрессирования).
 2. Обнаружить выбросы в столбце `bmi`.

3. Создать новый признак `bmi_age_interaction = bmi × age`.
4. Проверить наличие пропусков (их быть не должно).
5. Удалить дубликаты.

Вариант 3: Wine (качество вина)

- **Источник:** `from sklearn.datasets import load_wine`
- **Особенности данных:** Химический состав вин из трёх сортов.
- **Задачи:**
 1. Добавить в данные случайные пропуски (10% значений в столбце `alcohol`) для тренировки.
 2. Заполнить пропуски медианой по классу вина.
 3. Создать признак `total_phenols_flav = total_phenols + flavanoids`.
 4. Нормализовать все числовые признаки методом `min-max`.
 5. Проверить наличие выбросов в столбце `proline` методом `IQR`.

Вариант 4: Iris (цветки ириса)

- **Источник:** `sns.load_dataset('iris')`
- **Особенности данных:** Данные уже чистые, задача — создать новые признаки.
- **Задачи:**
 1. Создать признак `petal_area = petal_length × petal_width`.
 2. Создать признак `sepal_area = sepal_length × sepal_width`.
 3. Создать признак `shape_ratio = petal_length / sepal_length`.
 4. Сгруппировать данные по виду и добавить столбец со средним значением `petal_length` для каждого вида.
 5. Проверить данные на дубликаты (обычно их нет).

Вариант 5: Penguins (пингвины)

- **Источник:** `sns.load_dataset('penguins')`
- **Особенности данных:** Есть пропуски в столбцах `bill_length_mm`, `bill_depth_mm`, `sex`.
- **Задачи:**
 1. Обработать пропуски в числовых столбцах медианой по виду пингвина.
 2. Обработать пропуски в столбце `sex` модой (наиболее частым значением).

3. Создать признак `bill_area = bill_length_mm × bill_depth_mm`.
4. Удалить выбросы в `body_mass_g` методом IQR.
5. Проверить и удалить дубликаты.

Вариант 6: Planets (экзопланеты)

- **Источник:** `sns.load_dataset('planets')`
- **Особенности данных:** Пропуски в столбцах `mass`, `distance`.
- **Задачи:**
 1. Заполнить пропуски в `mass` медианой по методу обнаружения (`method`).
 2. Заполнить пропуски в `distance` медианой по году открытия.
 3. Создать признак `discovery_decade` (1990-е, 2000-е, 2010-е).
 4. Удалить строки, где `orbital_period < 0`.
 5. Удалить дубликаты.

Вариант 7: Diamonds (алмазы)

- **Источник:** `sns.load_dataset('diamonds')`
- **Особенности данных:** Большой датасет (около 54000 строк) с категориальными признаками.
- **Задачи:**
 1. Закодировать категориальные столбцы (`cut`, `color`, `clarity`) в числовые (например, по качеству: Fair=1, Good=2, Very Good=3, Premium=4, Ideal=5).
 2. Создать признак `volume = x × y × z`.
 3. Удалить строки с нулевым или отрицательным объёмом.
 4. Обработать выбросы в `price` методом IQR (удалить).
 5. Проверить и удалить дубликаты.

Вариант 8: Tips (чаевые)

- **Источник:** `sns.load_dataset('tips')`
- **Особенности данных:** Небольшой датасет, почти без пропусков.
- **Задачи:**
 1. Создать признак `tip_percentage = (tip / total_bill) × 100`.
 2. Создать категориальный признак `group_size` = small (<3 человек), medium (3-5), large (>5).
 3. Обнаружить выбросы в `tip_percentage` методом IQR и удалить их.
 4. Добавить случайные пропуски (5%) в столбце `day` и заполнить их модой.

5. Создать признак `is_weekend = True`, если `day` в ['Sat', 'Sun'].

Вариант 9: MPG (расход топлива)

- **Источник:** `sns.load_dataset('mpg')`
- **Особенности данных:** Пропуски в `horsepower`.
- **Задачи:**
 1. Заполнить пропуски в `horsepower` медианой по стране происхождения (`origin`).
 2. Создать признак `power_to_weight = horsepower / weight`.
 3. Нормализовать столбец `displacement` методом `min-max`.
 4. Преобразовать столбец `origin` в числовой (1, 2, 3) или `one-hot-encoding`.
 5. Удалить дубликаты.

Вариант 10: Flights (авиаперевозки)

- **Источник:** `sns.load_dataset('flights')`
- **Особенности данных:** Временной ряд, задача — подготовка для прогнозирования.
- **Задачи:**
 1. Преобразовать столбцы `year` и `month` в единый столбец `date` типа `datetime`.
 2. Создать признаки: `month_num` (число месяца), `season` (зима, весна, лето, осень).
 3. Заполнить пропуски (если есть) методом `forward fill (ffill)`.
 4. Создать лаговые признаки: `passengers_lag1` (значение за предыдущий месяц) с помощью `shift()`.
 5. Удалить дубликаты.

Контрольные вопросы для самопроверки

1. Почему этап очистки данных занимает до 80% времени в проектах Data Science?
2. В каких случаях лучше удалить строки с пропусками, а в каких — заполнить их?
3. Чем отличается `fillna(df.mean())` от `fillna(df.median())` и когда какой метод предпочтительнее?
4. Что такое параметр `keep='last'` в методе `drop_duplicates()`?
5. Как работает метод обнаружения выбросов IQR? Что означают границы $Q1 - 1.5 * IQR$ и $Q3 + 1.5 * IQR$?
6. В каких случаях выбросы нельзя удалять и почему?
7. Для чего используется `groupby().transform()` при заполнении пропусков?
8. Какие новые признаки можно создать в датасете о транзакциях интернет-магазина?
9. Почему после предобработки важно выполнять валидацию данных?
10. Что произойдёт, если не обработать дубликаты перед анализом или обучением модели?

Дополнительные источники информации и литература

1. Документация pandas по обработке пропусков: https://pandas.pydata.org/docs/user_guide/missing_data.html
2. Документация по drop_duplicates: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.drop_duplicates.html#reference:13
3. Документация по fillna: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.fillna.html#reference:14>
4. Руководство по обнаружению и обработке выбросов в Pandas: <https://www.kdnuggets.com/how-to-handle-outliers-in-dataset-with-pandas#reference:15>
5. Уэс Маккинни «Python и анализ данных» (глава 7 — очистка и подготовка данных)
6. Курс по очистке данных на Kaggle: <https://www.kaggle.com/learn/data-cleaning>
7. Data Wrangling: полное руководство: <https://docs.kanaries.net/articles/data-wrangling#reference:16>
8. Шпаргалка по pandas для предобработки данных (KDNuggets): <https://www.kdnuggets.com/10-essential-pandas-commands-data-preprocessing#reference:17>

9. Критерии оценивания (100-балльная шкала)

Критерий	Баллы	Пояснение
1. Этап Discover (обнаружение)	10	
- Загрузка данных (<code>read_csv</code> или <code>load_dataset</code>)	3	Правильный выбор источника
- Первичный осмотр (<code>.info()</code> , <code>.head()</code> , <code>.describe()</code>)	4	Все три метода применены
- Выявление проблем (пропуски, дубликаты, выбросы)	3	Краткий анализ проблем в Markdown
2. Этап Structure (структурирование)	10	
- Переименование столбцов (<code>.rename()</code>)	3	Осмысленные имена
- Преобразование типов (<code>astype()</code> , <code>pd.to_datetime()</code>)	4	Корректное приведение типов
- Настройка индекса (если необходимо)	3	
3. Этап Clean (очистка)	40	
- Обработка пропусков (выбор метода и его реализация)	15	Обоснованный выбор + корректный код
- Удаление дубликатов	10	Правильное использование параметров
- Обработка выбросов (обнаружение + действие)	15	Метод IQR + решение (удалить/оставить/ограничить)
4. Этап Enrich (обогащение)	20	
- Создание минимум 2 новых признаков	15	Осмысленность признаков
- Применение <code>apply</code> , <code>lambda</code> , <code>groupby</code> или <code>cut</code>	5	Использование хотя бы двух методов
5. Этап Validate (валидация)	10	
- Проверка отсутствия пропусков после очистки	3	<code>.isnull().sum()</code>
- Проверка корректности диапазонов	3	Проверка границ для числовых признаков
- Сохранение очищенного датасета	4	<code>.to_csv()</code> с правильным именем
6. Оформление и код	10	
- Код читаемый, с комментариями	5	
- Markdown-ячейки с пояснениями	5	Описание каждого этапа
ИТОГО	100	

Шкала перевода баллов в оценку

- **90–100 баллов** — отлично (все этапы выполнены корректно, обоснованы решения, созданы полезные признаки)
- **75–89 баллов** — хорошо (небольшие недочёты, например, не все выбросы обработаны)
- **60–74 баллов** — удовлетворительно (пропущен один из этапов или допущены существенные ошибки)
- **менее 60 баллов** — неудовлетворительно (требуется повторное выполнение).